Computational PDEs: Classical Approaches and Physics-Informed Learning

Day 1 — ODEs, Poisson Equations, Fourier Methods, and PINNs

Hiroyuki Miyoshi (The University of Tokyo)

9 November 2026

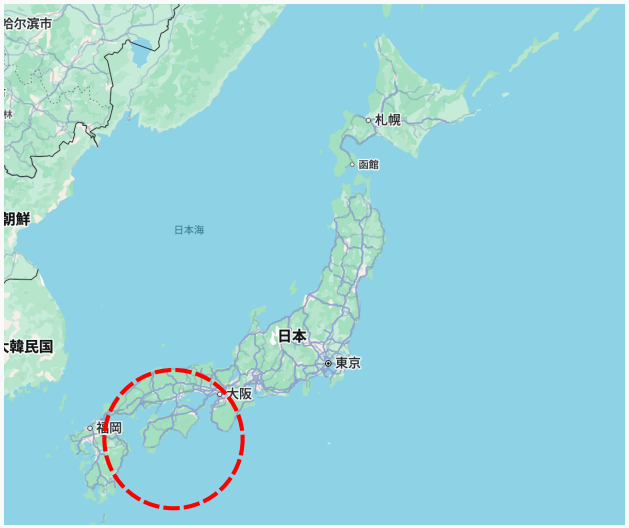
The University of Tokyo
ELTE Erasmus+ ICM Teaching Mobility
Faculty of Informatics, Eötvös Loránd University

Introduction / Jó reggelt!

Self-introduction

Hiroyuki Miyoshi / 三好　裕之/みよし　ひろゆき/ ミヨシ　ヒロユキ

(born in 1994 in Shikoku, Japan)









Self-introduction

Hiroyuki Miyoshi (born in 1994 in Shikoku, Japan)

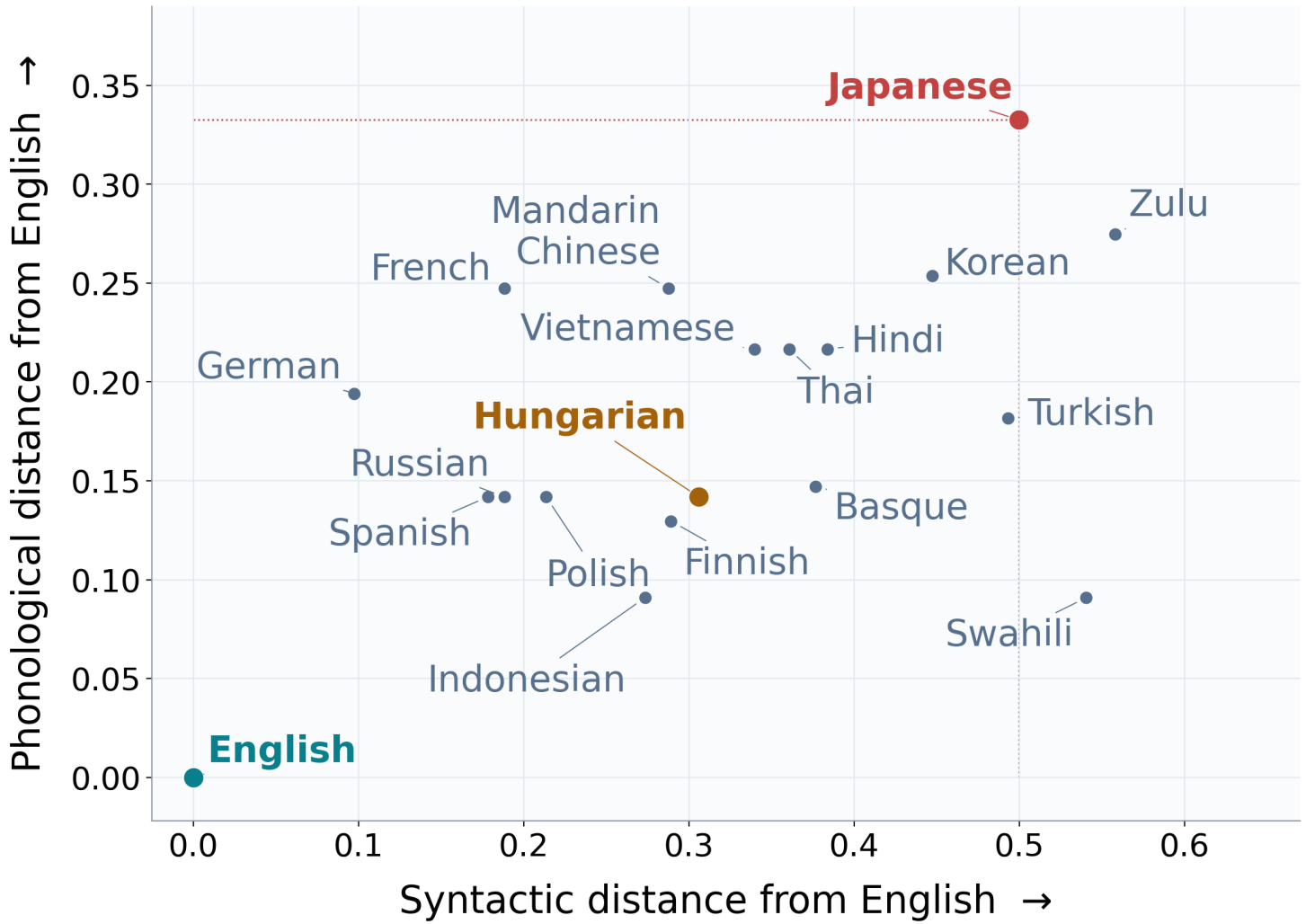
Research Background

- April 2017 – March 2019:
  Master's Program, Department of Information Physics and Computing, The University of Tokyo
- October 2019 – September 2023:
  Department of Mathematics, Imperial College London
- October 2023 – March 2025:
  Postdoctoral Researcher, The University of Tokyo
- April 2025 – present:
  Assistant Professor, The University of Tokyo

Research interest

- **Complex analysis** in **multiply connected domains**, **complex fluid dynamics** $\rightarrow$ Computational PDEs

English vs Japanese

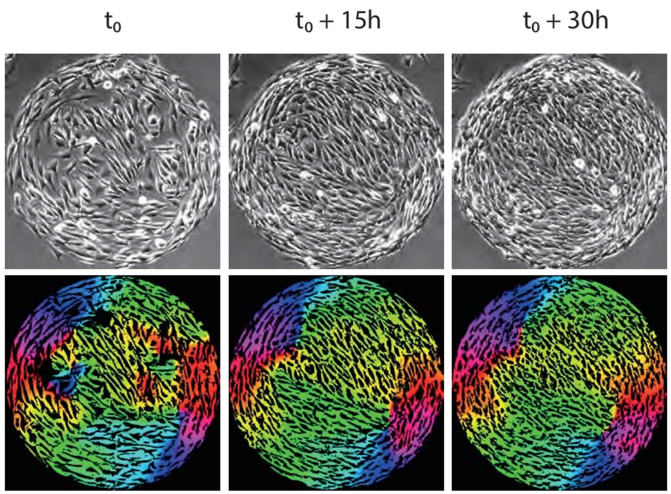


Course Objectives (four mornings)

Differential equations appear in many fields, and each one depends on an operator, a domain, and boundary constraints.



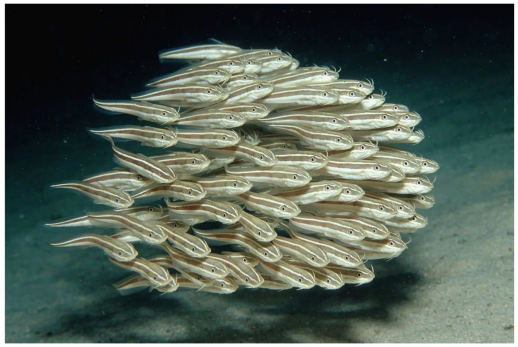
Tropical cyclone



Cell culture experiment



Heat sink



A school of fish

Course Objectives (four mornings)

Differential equations appear in many fields, and each one depends on an operator, a domain, and boundary constraints.

Course Aim

- Learn standard forms of simple PDEs
- Learn solution methods via classical approaches (complex analysis)
  $\rightarrow$ How to solve PDEs using complex analysis?
- Learn solution methods via physics-informed neural networks

Github repository

Available at

`https://github.com/wp1306/Comp_PDEs_Erasmus`.

Software (recommended)

On your laptop

- **Visual Studio Code** (`https://code.visualstudio.com/`)
- I am using **Cursor** (AI-powered IDE) (`https://www.cursor.com/`)
- VS Code extensions: **Python**, **Jupyter** (Microsoft)
- Python env `comp_pde` (conda or venv; Python **3.11**)

Notebooks: `work/pinn_learning/` (e.g. `01_ode_pinn.ipynb`).

Setup guide (English): `work/pinn_learning/ENV_SETUP.md`.

Day 1 goals (1/2)

We assume

- Python (NumPy, Matplotlib), basic linear algebra
- Terminology: derivative, boundary condition
- Basic knowledge of differential equations

We do not assume

- Fluid/physics modelling courses
- Prior PyTorch experience (introduced today)
- Prior knowledge of complex analysis

Monday goals

After today you should be able to

1. write $u' + u = 0$ as a residual;
2. name the four pieces of the stored network: architecture, derivative, loss, step;
3. compute one sine-series coefficient on a rectangle;
4. quote a relative L^2 error of the network against the series.

Pipeline

Data (domain, operator, boundary constraints) $\rightarrow$ **Algorithm** (series or network) $\rightarrow$ **Code** $\rightarrow$ **Check** (relative L^2)

ODE warm-up

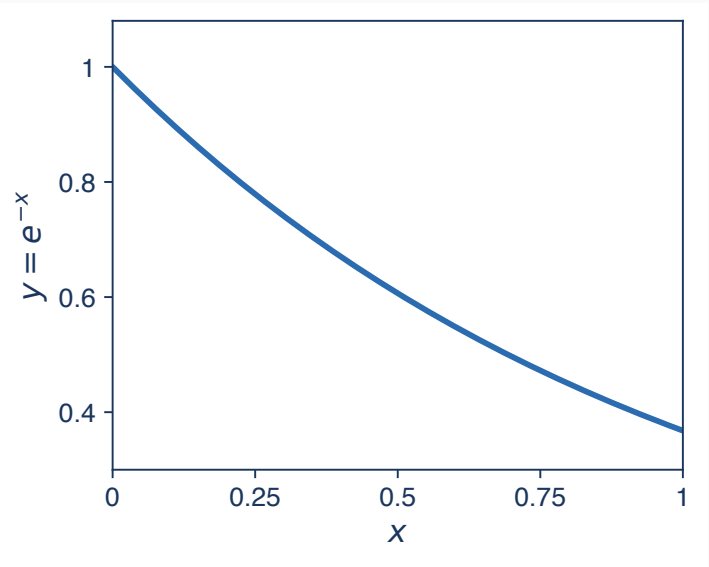
1D ODE: Boundary Value Problem (1/2)

Consider $x \in (0, 1)$:

$$u'(x) + u(x) = 0, \qquad u(0) = 1. \tag{1}$$

where $' := \mathrm{d}/\mathrm{d}x$.
Exact solution: $u(x) = e^{-x}$.

$$\begin{aligned} \because \quad & \frac{\mathrm{d}u}{u} = -\mathrm{d}x \\ \Rightarrow & \log u = -x + C \\ \Rightarrow & u = e^{-x+C}. \end{aligned} \tag{2}$$



The solution is uniquely determined by the boundary condition.

← How about the approach of **Physics-informed learning (PINNs)**?

PINNs are a type of **neural network** that is trained to solve PDEs.

1D ODE: residuals (2/2)

Introduce the **PDE residual**

$$r(x) := u'(x) + u(x). \tag{3}$$

Valid solution $\Leftrightarrow r(x) = 0$ in $(0, 1)$ and $u(0) = 1$.

- **Input**: interval, differential operator, boundary data
- **Output**: values $\{u(x_i)\}$ or a surrogate $u_\theta(x)$
- **Check**: compare with e^{-x}

This is exactly what PINNs penalise:

$$\mathcal{L}_{\text{ODE}} = \text{MSE}(r^2), \qquad \mathcal{L}_{\text{BC}} = |u(0) - 1|^2. \tag{4}$$

Loss function:

$$\mathcal{L} = \mathcal{L}_{\text{ODE}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}. \tag{5}$$

PINN idea (1/8)

Physics-Informed Neural Network (PINN): approximate $u \approx u_\theta$ and minimise

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PDE}}(\theta) + \lambda_{\mathrm{BC}}\, \mathcal{L}_{\mathrm{BC}}(\theta). \tag{6}$$

- $\mathcal{L}_{\mathrm{PDE}}$: mean squared PDE residual at **collocation points**
- $\mathcal{L}_{\mathrm{BC}}$: mean squared boundary error
- λ_{BC}: weight balancing interior vs. boundary (often needs tuning)

No mesh: sample points $x_i \in (0, 1)$; derivatives from **autograd**, not FD on u_θ.

$$u'(x) \sim \frac{u(x+h) - u(x)}{h} \tag{7}$$

PINN vs classical solvers (2/8)

	Fourier / FD	PINN
Unknown	grid / mode coeffs	network weights θ
Linear algebra	yes ($A\boldsymbol{u} = \boldsymbol{f}$)	no
Optimisation	no	yes (Adam on θ)
Where PDE enters	discretisation	residual loss
Validation	same: vs. exact / manufactured u	same

PINN is an **optimisation-based surrogate**, not a direct linear solve.

MLP architecture (3/8)

We use a fully connected network (`pinn_utils.MLP`):

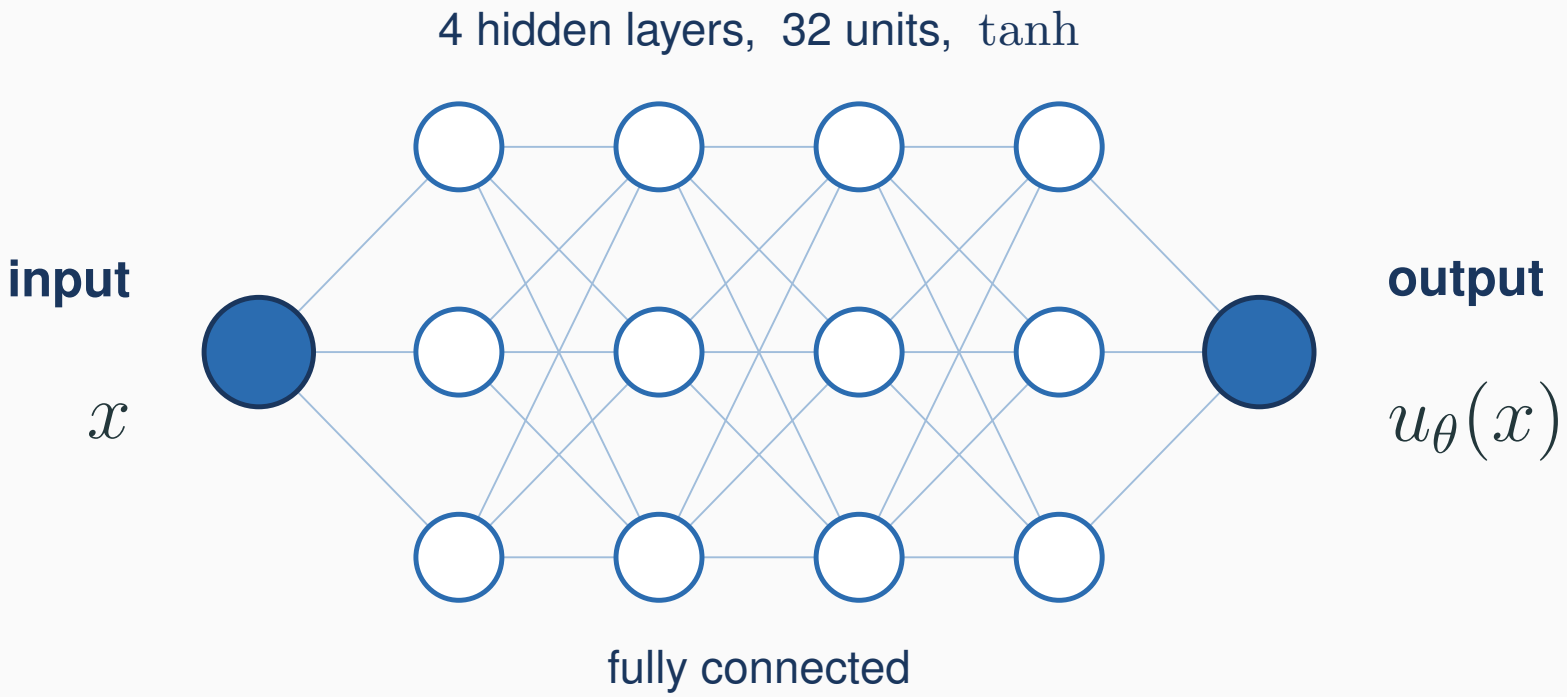
$$u_\theta(x) = W_L\,\sigma(\cdots\sigma(W_1 x + b_1)\cdots) + b_L, \qquad \sigma = \tanh\,. \tag{8}$$

Typical sizes in our code

- ODE (`01_ode_pinn`): `d_in=1`, `width=32`, `depth=4`
- 1D Poisson (`02_poisson_1d_pinn`): `width=64`, `depth=5`
- 2D Poisson: input (x, y), same MLP pattern

Smooth $\tanh$ activations help stable high-order derivatives via autograd.

Fully connected MLP



The network takes the coordinate x and returns the value $u_\theta(x)$.

Each hidden layer has 32 units.

Automatic differentiation

To form u'' in the Poisson residual we need exact derivatives of the **network**:

$$u_\theta(x) \xrightarrow{\texttt{grad}} u'_\theta(x) \xrightarrow{\texttt{grad}} u''_\theta(x). \tag{9}$$

`pinn_utils.grad` wraps `torch.autograd.grad` with `create_graph=True` so second derivatives backpropagate into θ.

Key point

- Collocation points must have `requires_grad=True`.
- We do **not** approximate u'' by finite differences on the trained u_θ.

Training algorithm

Input: domain, PDE operator, BC data, hyperparameters
Output: trained u_θ, loss history, relative L^2 error

1. Sample collocation $\{x_i\}_{i=1}^{N_{\text{coll}}}$ and boundary points
2. Initialise MLP parameters θ; set optimiser (Adam, $\text{lr} = 10^{-3}$)
3. **For** `epoch = 1 ...EPOCHS`:
   - Forward: $u_\theta(x_i)$; compute u', u'' by autograd
   - Residual r_i (e.g. $u'_\theta + u_\theta$ or $-u''_\theta - f$)
   - $\mathcal{L}_{\text{PDE}} = \text{mean}(r_i^2), \quad \mathcal{L}_{\text{BC}} = \text{mean}(|u_\theta - g|^2)$ on boundary
   - $\mathcal{L} = \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}$
   - `loss.backward()`; `optimizer.step()`
4. Evaluate on test grid; compare with exact solution

Hyperparameters

Often sensitive

- λ_{BC}: too small $\Rightarrow$ BC violated
- N_{coll}: too few $\Rightarrow$ poor interior fit
- `width / depth`: capacity vs. overfitting
- `EPOCHS`, learning rate

What to monitor

- Loss curves (PDE vs. BC separately)
- Relative L^2 vs. exact
- Compare with FD / Fourier baseline

ODE PINN: code overview

First cell of `01_ode_pinn.ipynb`.

1. **Seed.** `set_seed(42)` fixes the PyTorch and NumPy random state.
2. **Network.** `MLP` maps $x \mapsto u_\theta(x)$: fully connected, $\tanh$, `width=32`, `depth=4`.
3. **Derivative.** `grad` calls `autograd.grad` with `create_graph=True`, so u'_θ can enter the loss.
4. **Check.** `relative_l2` compares u_θ with the exact solution e^{-x}.

How they connect

u_θ is the unknown. `grad` builds the residual $r = u'_\theta + u_\theta$. Training minimises $\mathcal{L}$;

PyTorch autograd

PyTorch records operations on tensors with `requires_grad=True`. That record is the computational graph.

1. **Input.** Collocation points x need `requires_grad_(True)`: we differentiate with respect to x.
2. **Forward.** The network is a composition of linear maps and $\tanh$. Autograd stores this composition.

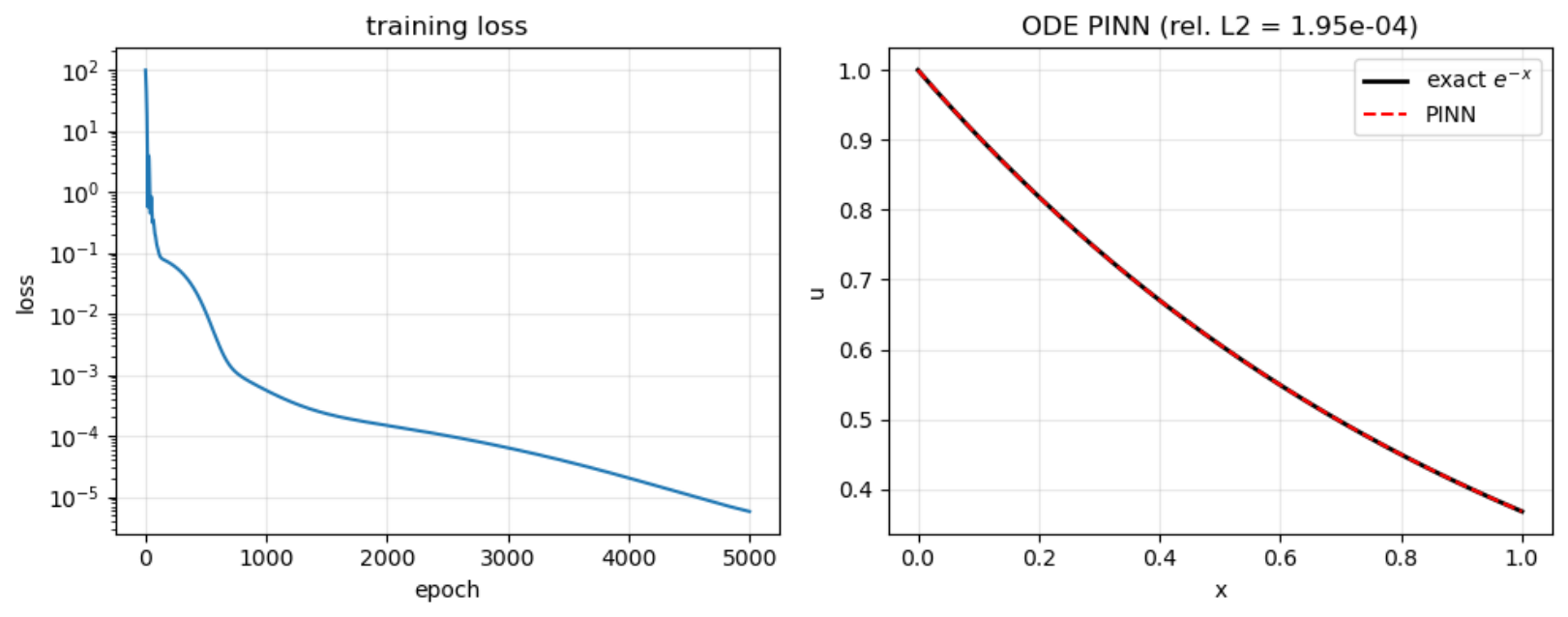
$$\frac{\mathrm{d}u}{\mathrm{d}x} = \frac{\partial u}{\partial w_1}\frac{\partial w_1}{\partial w_2}\cdots\frac{\partial w_n}{\partial x}. \tag{10}$$

3. **Derivative in x.** `autograd.grad`(u_θ, x) returns u'_θ. `create_graph=True` keeps u'_θ in the graph, so it still depends on θ.
4. **Derivative in θ.** `loss.backward()` computes $\partial\mathcal{L}/\partial\theta$ through both u_θ and u'_θ. Adam updates θ.

Training

Table 1: Training loss at selected epochs

Epoch	$\mathcal{L}$ (total loss)	$\mathcal{L}_{\text{PDE}}$	$\mathcal{L}_{\text{BC}}$
1000	1.845×10^{-3}	1.845×10^{-3}	3.747×10^{-9}
2000	5.523×10^{-4}	5.522×10^{-4}	3.869×10^{-10}
3000	8.943×10^{-5}	8.943×10^{-5}	3.769×10^{-11}
4000	3.486×10^{-5}	3.486×10^{-5}	3.638×10^{-12}
5000	1.979×10^{-5}	1.979×10^{-5}	2.988×10^{-12}



Modifications

Modifications 1 (`01_ode_pinn2.ipynb`)

$$u'(x) + u(x) = \frac{1}{1+x}, \qquad u(0) = 1. \tag{11}$$

(i) Exact solution: (special function)

$$u(x) = e^{-x}\left(1 + e^{-1}(\mathrm{Ei}(x+1) - \mathrm{Ei}(1))\right). \tag{12}$$

where $\mathrm{Ei}(x)$ is the **exponential integral function**:

$$\mathrm{Ei}(x) := \int_{-\infty}^{x} \frac{e^t}{t} \mathrm{d}t. \tag{13}$$

(ii) PINNs solution: change the residual to $r(x) := u' + u - 1/(1+x)$.

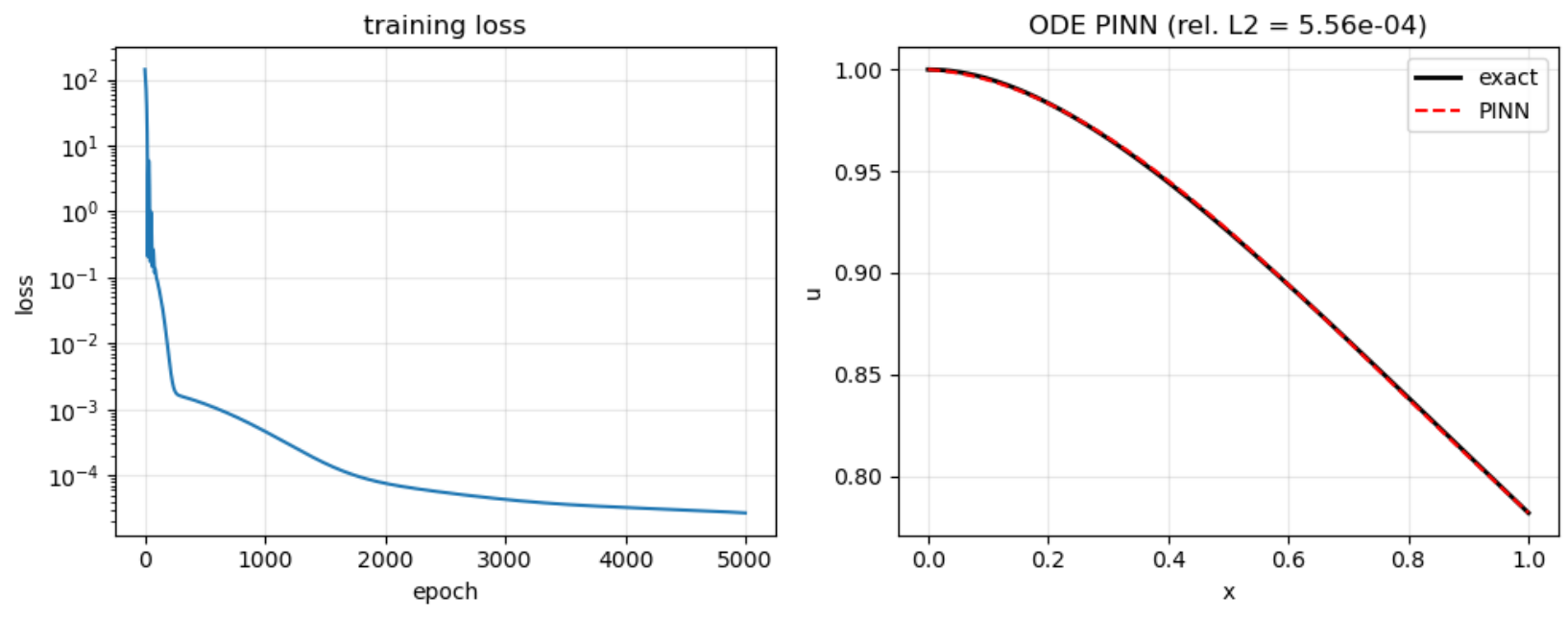


Figure 1: Result of PINN solution for the modified ODE problem.

PINNs can solve the modified ODE problem by changing the residual.

Modifications 2 (`01_ode_pinn3.ipynb`)

$$u'(x) + u(x) = \sin(kx), \qquad u(0) = 1. \tag{14}$$

(i) Exact solution: (not so difficult to derive)

$$u(x) = \frac{1 + k + k^2}{1 + k^2} e^{-x} + \frac{\sin(kx) - k\cos(kx)}{1 + k^2}. \tag{15}$$

(ii) PINNs solution: change the residual to $r(x) := u' + u - \sin(kx)$.

Easy to modify the problem to solve other problems!

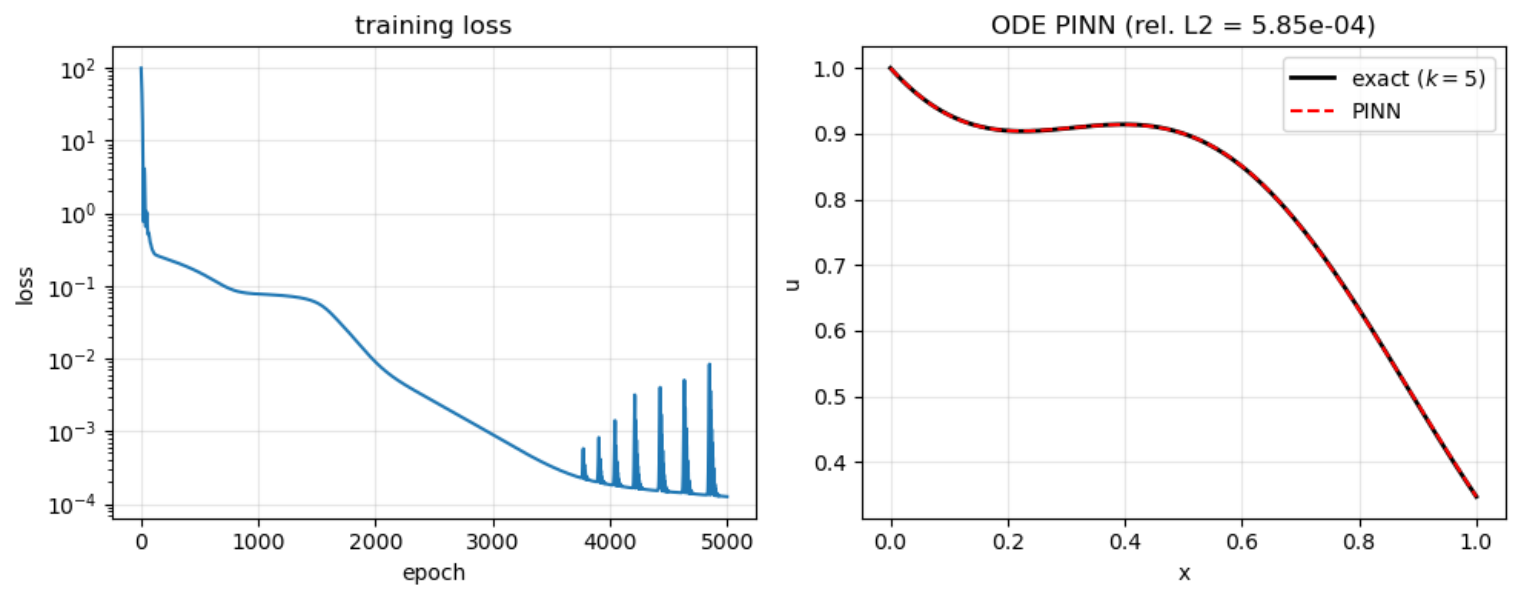


Figure 2: Result of PINN solution ($k = 5$ and $k = 30$). When k is large, the solution is not accurate.

How can we obtain the accurate solution?

- Increase the number of hidden layers
  (Network becomes more complex)
- Increase the number of units per layer
- Increase the number of collocation points (more data points)
- **Increase the epochs** (more training) –> Next page
- **Change the input data** –> If possible, works well!

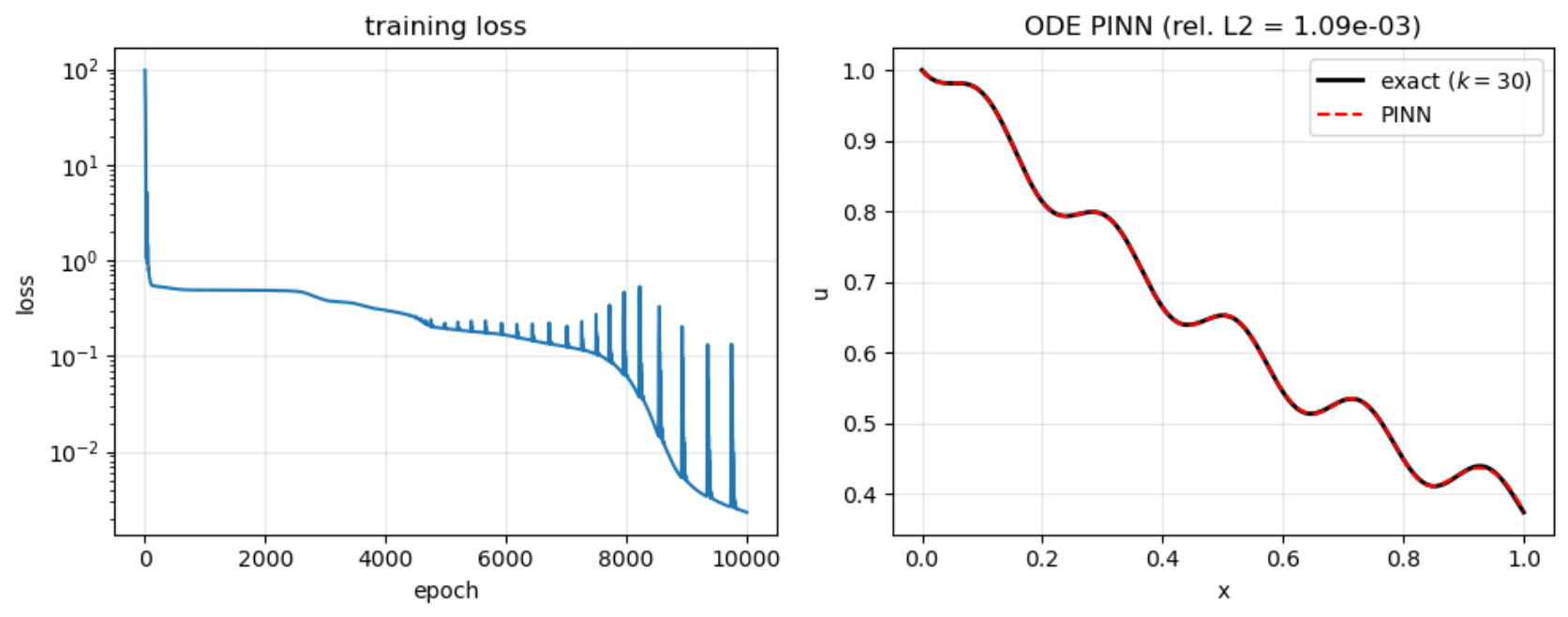


Figure 3: Result of PINN solution ($k = 30$) with the same initial guess.

Change the input data

$$u'(x) + u(x) = \sin(kx), \qquad u(0) = 1. \tag{16}$$

We consider the problem which satisfies $u'(x) + u(x) = 0$ with $u(0) = 1$:

$$u_{\text{guess}}(x) = e^{-x} \tag{17}$$

and

$$u(x) = u_{\text{guess}}(x) + x v_\theta(x). \tag{18}$$

Use PINNs to find $v_\theta(x)$.

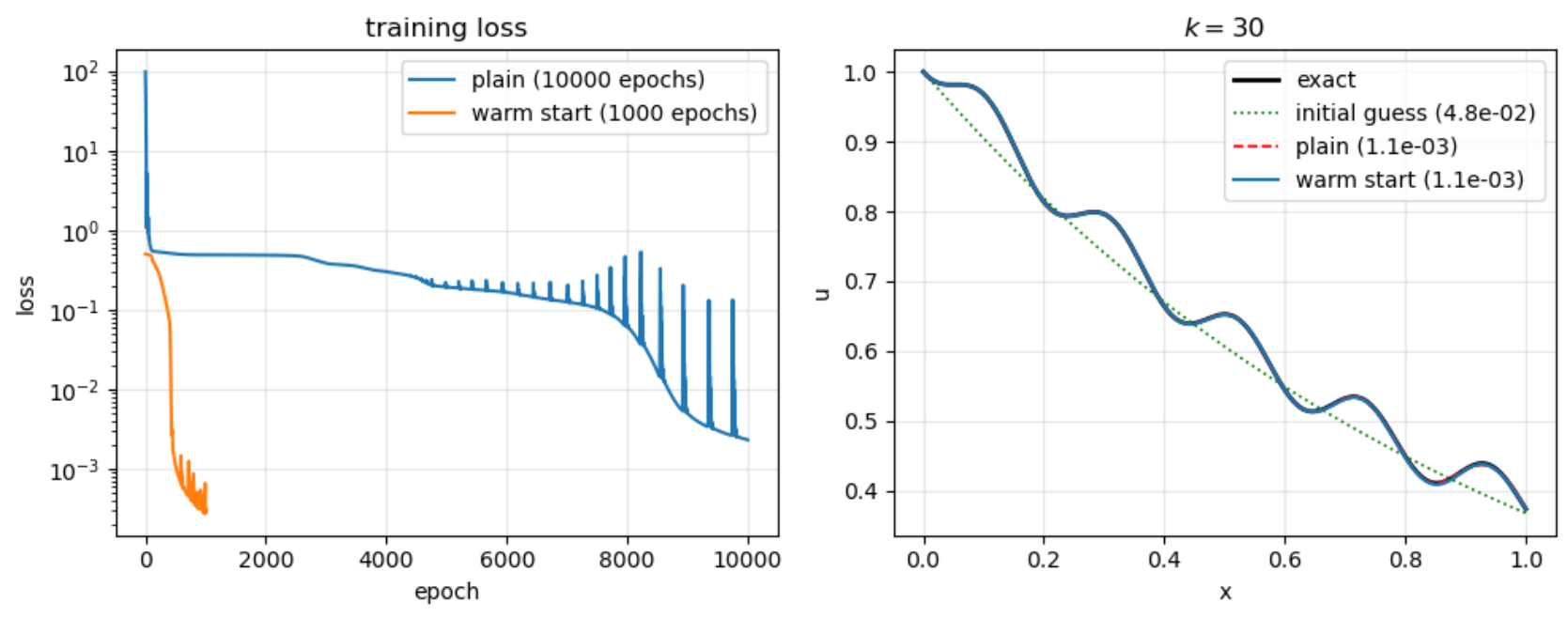


Figure 4: Result of PINN solution ($k = 30$) with the warm start initial guess.

Summary 1

- PINNs are powerful tools for solving PDEs.
- Easy to modify the problem to solve other problems!
- However, there are some limitations to obtain the accurate solution.

Good initial guess is important to obtain the accurate solution.

- 1D problems $\rightarrow$ 2D problem

2D problems

From ODE to 2D PDE (1/2)

We have already seen 1D problems. Now let's consider 2D problems.

Many simulations need a field $u(x, y)$ on $\Omega \subset \mathbb{R}^2$.

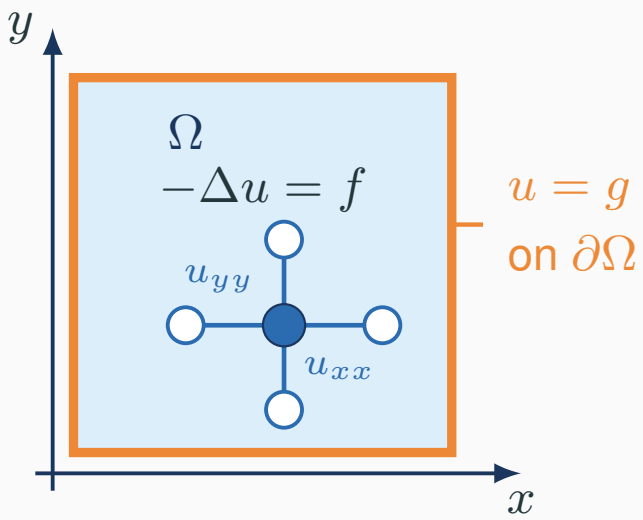
Poisson equation

$$-\Delta u = f \qquad \text{in } \Omega, \tag{19}$$

$$u = g \qquad \text{on } \partial\Omega, \tag{20}$$

where

$$\Delta := \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}. \tag{21}$$



- Unknown data on a **2D grid**
- Neighbours are coupled (global influence of boundaries)

1D Poisson benchmark (2/2)

Consider 1D problem first.

On $(0, 1)$ with $u(0) = u(1) = 0$:

$$-u''(x) = f(x). \tag{22}$$

Course benchmark (`02_poisson_1d_pinn`):

$$f(x) = \pi^2 \sin(\pi x). \tag{23}$$

The solution is

$$u(x) = \sin(\pi x). \tag{24}$$

Fourier sine series in 1D (1/3)

Consider 1D problem first.

On $(0,1)$ with $u(0) = u(1) = 0$:

$$-u''(x) = f(x). \tag{25}$$

For (22) with homogeneous Dirichlet BCs, expand

$$u(x) = \sum_{n=1}^{\infty} u_n \sin(n\pi x). \tag{26}$$

Then $-u''(x) = \sum_n (n\pi)^2 u_n \sin(n\pi x)$ and

$$f(x) = \sum_{n=1}^{\infty} f_n \sin(n\pi x), \qquad f_n = 2\int_0^1 f(x)\sin(n\pi x)\,\mathrm{d}x. \tag{27}$$

Fourier sine series in 1D (2/3)

Mode matching gives a **diagonal linear system**:

$$(n\pi)^2 u_n = f_n \qquad \Longrightarrow \qquad u_n = \frac{f_n}{(n\pi)^2}. \tag{28}$$

For $f(x) = \pi^2 \sin(\pi x)$: only $n = 1$ is active, $u_1 = 1$.

In code: truncate at $N_{\max}$, evaluate $u_N(x) = \sum_{n=1}^{N_{\max}} u_n \sin(n\pi x)$, plot error vs. $N_{\max}$.

Fourier sine series in 2D (3/3)

On $(0,1)^2$ with $u|_{\partial\Omega} = 0$:

$$u(x,y) = \sum_{m,n\geq 1} u_{mn} \sin(m\pi x)\sin(n\pi y), \tag{29}$$

$$\big((m\pi)^2 + (n\pi)^2\big) u_{mn} = f_{mn}, \qquad f_{mn} = 4\int_0^1\int_0^1 f \sin(m\pi x)\sin(n\pi y)\,\mathrm{d}x\,\mathrm{d}y. \tag{30}$$

Benchmark: only $(m,n) = (1,1)$ contributes.

Links to FFT / signal processing and diagonalisation in mode space.

Example (i) of 2D Poisson equation

Unit square, zero boundary values:

$$-\Delta u = 1 \quad \text{in } (0,1)^2, \qquad u = 0 \quad \text{on } \partial\Omega. \tag{31}$$

Sine coefficients of $f \equiv 1$ vanish unless both m and n are odd:

$$f_{mn} = \frac{16}{mn\pi^2}, \qquad u_{mn} = \frac{16}{mn\pi^4(m^2+n^2)}. \tag{32}$$

Leading term $(m,n) = (1,1)$: $u_{11} = 8/\pi^4$. The next terms $(1,3)$ and $(3,1)$ are smaller by a factor 15.

In code: truncate the double sum and plot the partial sum.

Example (ii) of 2D Poisson equation

Same square and boundary conditions. Put two modes of equal size into f:

$$f(x,y) = \sin(\pi x)\sin(\pi y) + \sin(8\pi x)\sin(8\pi y). \tag{33}$$

Only $(1,1)$ and $(8,8)$ are active, and

$$u_{11} = \frac{1}{2\pi^2}, \qquad u_{88} = \frac{1}{128\pi^2} = \frac{u_{11}}{64}. \tag{34}$$

$$u = \frac{\sin(\pi x)\sin(\pi y)}{2\pi^2} + \frac{\sin(8\pi x)\sin(8\pi y)}{128\pi^2}. \tag{35}$$

f shows both waves equally. In u the fast wave is 64 times smaller: division by $m^2 + n^2$ damps it. Plot f and u side by side.

When to use Fourier methods

Good fit

- Rectangular domains
- Constant coefficients
- Simple boundary types
- Smooth u: fast mode decay

Limitations

- Complex geometry
- Variable coefficients
- Nonlinear PDEs

$\Rightarrow$ motivates **analytical methods** and **PINNs** (collocation + optimisation).

Three methods compared

Method	Discretisation	Linear solve?	Optimisation?
Fourier sine series	modes (m, n)	yes (diagonal)	no
Analytical methods	no	no	no
PINN	collocation	no	yes (Adam)

PINNs

PINN for 2D Poisson (preview)

Residual on $\Omega = (0,1)^2$:

$$r(x,y) = -u_{xx} - u_{yy} - f(x,y), \qquad f = 2\pi^2 \sin(\pi x)\sin(\pi y). \tag{36}$$

Collocation in the interior; boundary samples on all four edges.

See `03_poisson_2d_pinn`; optional warm start in `04_poisson_2d_pinn_warmstart`.

Always report **relative L^2 error** (`pinn_utils.relative_l2`).

Remark

- PINNs are powerful tools for solving PDEs.
- However, there are some limitations to obtain the accurate solution.

For 2D problems, analytical approach called complex analysis is one of the most powerful tools.

We will cover this in the next lecture.

Summary

Three methods compared

Method	Discretisation	Linear solve?	Optimisation?
Fourier sine series	modes (m, n)	yes (diagonal)	no
Finite differences	grid	yes (sparse A)	no
PINN	collocation	no	yes (Adam)

- Fourier: fast when geometry is simple; connects to FFT
- FD: general grids; explicit matrix structure
- PINN: mesh-free; optimisation and validation matter

When classical methods work, and when a PINN is usable

Classical methods

- Linear PDE you can discretise
- Rectangle (sine series) or a grid (FD)
- BCs that match the basis or stencil
- Named error: truncation or $O(h^2)$

A PINN is usable when

- Residual and BCs are easy to code
- A mesh or eigenbasis is awkward
- You will tune λ_{BC}
- A baseline exists for relative L^2

On today's benchmarks

Fourier and FD are the accurate solvers. Train a PINN on the same problem to compare; it is a surrogate, not a replacement.

Monday, in one pass

A stored network for $u' + u = 0$: its cells, and one run where a poor initialisation fails. A sine series for a Poisson equation on a rectangle: one division per mode. The check is a relative L^2 error. Two short exercises (about 15–20 minutes in total). Tuesday does not train a network.

Exercises

Exercises (15–20 min)

Format

Two questions. Work in pairs if you like. Use `01_ode_pinn.ipynb` for Question 2 (kernel `comp_pde`). Write answers in a note or a Markdown cell.

1. **Residual and exact solution** (about 7–8 min) — pen and paper.
2. **Relative L^2 and the BC weight** (about 8–10 min) — short notebook run.

Software references

- `work/pinn_learning/ENV_SETUP.md` — VS Code, `comp_pde`, install steps
- `work/pinn_learning/requirements-comp_pde.txt` — pinned dependencies
- `work/pinn_learning/pinn_utils.py` — `MLP`, `grad`, `relative_l2`
- `work/pinn_learning/01_ode_pinn.py`
- `work/pinn_learning/02_poisson_1d_pinn.py`
- `work/pinn_learning/03_poisson_2d_pinn.ipynb`

Homework (about 1 hour)

- **Notebook:** `01_ode_pinn3.ipynb`
  $u' + u = \sin(kx)$, $u(0) = 1$ (exact u is in the notebook).
- **Deadline:** 18 November 2026, 23:59 (Budapest time).
- **Submit:** PDF/HTML of the notebook, or a short PDF:
  `https://drive.google.com/drive/folders/1bssVFArYRS6jrQvduRfxj4lkiihmksX_?usp=sharing`

Tasks

1. **Plain PINN.** Train for $k = 5$ and $k = 30$. Record relative L^2; keep both plots.
2. **Warm start ($k = 30$).** Set `USE_FULL_GUESS = True`. Train $u = u_{\text{guess}} + x\, v_\theta$. Compare plain vs. warm relative L^2.
3. **Write-up (5–8 sentences).** Why is large k hard? Why does the warm start help?

Homework — what to include

Hand in

- A table with k, `USE_FULL_GUESS`, relative L^2
- The plots from Tasks 1–2
- The short write-up

File name: `FamilyName_GivenName_day1_hw`

Expected time

About one hour with the default `EPOCHS` in the notebook. If training is slow, you may use fewer epochs; state that in the table.